```java
// tanto al nodo siguiente (next) como al anterior (prev)
class Node  35 usages
{
    int value;  6 usages
    Node next;  25 usages
    Node prev;  15 usages

    Node(int value)  4 usages
    {
        this.value = value;
    }
}
```

```java
public DoubleLinkedList(int value)  1 usage
{
    Node newNode = new Node(value);
    this.length = 1;
    this.head = newNode;
    this.tail = newNode;
}

public Node getHead()  1 usage
{
    return this.head;
}

// Asigna manualmente el nodo cabeza
public void setHead(Node head)  no usages
{
    this.head = head;
}

// Asigna manualmente el nodo cola
public void setTail(Node tail)  no usages
{
    this.tail = tail;
}
```

```java
// Regresa el último nodo (o null si la lista está vacía)
public Node getTail()  1 usage
{
    return this.tail;
}

// Regresa la cantidad de nodos de la lista
public int getLength()  no usages
{
    return this.length;
}

// Si la lista está vacía, el nuevo nodo es head y tail.
// Si no, se enlaza en ambos sentidos con tail: tail.next -> nuevo, nuevo.prev -> tail.
public void append(int value)
{
    Node newNode = new Node(value);
    if (length == 0)
    {
        this.head = newNode;
        this.tail = newNode;
    }
    else
    {
        this.tail.next = newNode;
        newNode.prev = this.tail;
        this.tail = newNode;
    }
    length++;
}
```

```java
// Si la lista está vacía, el nuevo nodo es head y tail.
// Si no, se enlaza en ambos sentidos con head: nuevo.next -> head, head.prev -> nuevo.
public void prepend(int value)  2 usages
{
    Node newNode = new Node(value);
    if (length == 0)
    {
        this.head = newNode;
        this.tail = newNode;
    }
    else
    {
        newNode.next = this.head;
        this.head.prev = newNode;
        this.head = newNode;
    }
    length++;
}
```

```java
// Regresa false si el índice es inválido (menor que 0 o mayor que length).
public boolean insert(int index, int value)  2 usages
{
    if (index < 0 || index > length) return false;
    if (index == 0)
    {
        prepend(value);
        return true;
    }
    if (index == length)
    {
        append(value);
        return true;
    }
    Node newNode = new Node(value);
    Node before = getNode( index: index - 1);
    Node after = before.next;

    newNode.prev = before;
    newNode.next = after;
    before.next = newNode;
    after.prev = newNode;
    length++;
    return true;
}
```

```java
// head avanza al siguiente nodo, y a ese nuevo head se le borra el prev.
// El nodo removido se desconecta por completo. Si la lista queda vacía, tail = null.
public Node removeFirst()  2 usages
{
    if (length == 0) return null;
    Node temp = this.head;
    if (length == 1)
    {
        this.head = null;
        this.tail = null;
    }
    else
    {
        this.head = this.head.next;
        this.head.prev = null;
        temp.next = null;
    }
    length--;
    return temp;
}
```

```java
public Node removeLast()  2 usages
{
    if (length == 0) return null;
    Node temp = this.tail;
    if (length == 1)
    {
        this.head = null;
        this.tail = null;
    }
    else
    {
        this.tail = this.tail.prev;
        this.tail.next = null;
        temp.prev = null;
    }
    length--;
    return temp;
}
```

```java
public Node remove(int index)  1 usage
{
    if (index < 0 || index >= length) return null;
    if (index == 0) return removeFirst();
    if (index == length - 1) return removeLast();

    Node temp = getNode(index);
    temp.prev.next = temp.next;
    temp.next.prev = temp.prev;
    temp.next = null;
    temp.prev = null;
    length--;
    return temp;
}

// regresa el nodo en la posición "index" (null si es inválido).
// Optimización típica de la lista doble: si el índice está en la primera mitad
// se recorre desde head hacia adelante; si está en la segunda, desde tail hacia atrás.
public Node getNode(int index)  4 usages
{
    if (index < 0 || index >= length) return null;
    Node temp;
    if (index < length / 2)
    {
        temp = this.head;
        for (int i = 0; i < index; i++)
        {
            temp = temp.next;
        }
```

```java
public boolean setNode(int index, int value)  1 usage
{
    Node temp = getNode(index);
    if (temp == null) return false;
    temp.value = value;
    return true;
}

// "slow" avanza de uno en uno y "fast" de dos en dos. Cuando fast llega al final,
// slow está en la mitad. Si la cantidad de nodos es par, regresa el de la derecha.
public Node medio()  2 usages
{
    if (head == null) return null;
    Node slow = this.head;
    Node fast = this.head;
    while (fast != null && fast.next != null)
    {
        slow = slow.next;
        fast = fast.next.next;
    }
    return slow;
}
```

```java
public boolean esCiclo()  3 usages
{
    Node slow = this.head;
    Node fast = this.head;
    while (fast != null && fast.next != null)
    {
        slow = slow.next;
        fast = fast.next.next;
        if (slow == fast) return true;
    }
    return false;
}

// En cada nodo se intercambian sus referencias next y prev; después se
// intercambian head y tail. Tras el intercambio, el siguiente nodo a visitar
public void reversa()  1 usage
{
    Node current = this.head;
    Node temp = null;
    while (current != null)
    {
        temp = current.prev;
        current.prev = current.next;
        current.next = temp;
        current = current.prev;
    }
    temp = this.head;
    this.head = this.tail;
    this.tail = temp;
}
```

```java
// Se utiliza el método para detectar listas cicladas antes de poder imprimirla
public void printList()  3 usages
{
    if(!esCiclo())
    {
        Node temp = this.head;
        while (temp != null)
        {
            System.out.print(temp.value + " ");
            temp = temp.next;
        }
        System.out.println();
    }
    else
    {
        System.out.println("La lista está ciclada");
    }
}
}
```